Lugworm

Exercises

Jasper Vollmert
Institute of Digital and Autonomous Construction (IDAC)
Hamburg University of Technology
jasper.vollmert@tuhh.de

September 22, 2026

Abstract

This course teaches Grasshopper and the Lugworm plugin together, entirely inside Grasshopper. No printer is required for any exercise. Each session adds one or two components or inputs to what is already known, using geometry built in Grasshopper or generated with `Test Geometry`. Every result is read from component outputs: `info`, `preview`, `travels`, `layers`, `length`, `volume`, `time`, `reachOK`, and `problems`.

The course builds toward one shared final project that combines everything learned and is printed together once every session is complete. When the course is taught to a group, each session fills one lecture or lab block. Companion documents: *beginner-guide.tex* explains each component in plain language, and *reference.tex* explains the reasoning and serves as a lookup during class.

Contents

1 Structure of each exercise 3

2 Session 0: Wiring the pipeline 3

3 Session 1: Layer height, walls, and layer spacing 3

4 Session 2: Infill patterns side by side 4

5 Session 3: Vase mode (spiralize) 4

6 Session 4: Seam control 4

7 Session 5: Flow and pressure basics 5

8 Session 6: Overhang analysis 5

9 Session 7: Print Stats as a pre-flight check 6

10 Session 8: Non-planar modes (advanced) 6

11 Session 9: Robot simulation (dry run) 6

12 Session 10: A first pass through calibration (DoE) 7

13 Optional exercises 7

14 Final project (printed together) **7**

1 Structure of each exercise

Every exercise has the same parts: **Grasshopper concept**, **Goal**, **Build**, **Try this**, **Read the output**, and **Check yourself** (a pass or fail criterion that can be verified without a printer).

2 Session 0: Wiring the pipeline

Grasshopper concept: finding plugin components on the **Lugworm** tab; Panels; Heavy components and the manual compute button.

Goal: get geometry through `Slice`, `Lugworm Place`, and `G-code Generator` once, so that every later exercise only changes numbers.

Build:

1. Place an `Test Geometry` component (tab **Lugworm**, panel **6 Calibrate**). Right-click the component and choose **Cylinder** from the dropdown. Set `height = 80` and `radius = 40`.
2. Place `Slice` (**1 Slice**), `Place` (**2 Place**), and `G-code Generator` (**3 G-code**).
3. Connect `Test Geometry.geometry` to `Slice.geo`, `Slice.contours` to `Place.contours`, and `Place.moved` to `G-code Generator.crvs`.
4. Attach **Panels** to `Slice.info`, `G-code Generator.info`, and `G-code Generator.gcode`. A panel on `gcode` shows one line per item. For a large job, prefer the `info` summary or save the file, because a panel holding an entire print is slow to draw.
5. Note that `Slice` and `G-code Generator` each show a small compute button in the lower-right corner. Both are Heavy components (see *reference.tex* §2). Click each button once.

Try this: leave every input at the default for now and click both compute buttons.

Read the output: `Slice.info` should report a layer count and no per-object errors. `Lugworm G-code Generator.gcode` should show G-code that starts with a header comment block.

Check yourself: `Slice.layers` multiplied by `layerHeight` (default `3.0`) should be roughly the cylinder height of `80`, within one layer. If `layers` is `0`, check the wiring and remember to click the compute button after every change. Heavy components do not refresh on their own.

3 Session 1: Layer height, walls, and layer spacing

Grasshopper concept: Number Sliders; clicking the compute buttons again after a slider change.

Goal: understand how `layerHeight`, `wallCount`, and `spacing` change the sliced result, and connect that to the physical layer size on `G-code Generator`.

Try this:

1. Set `Slice.layerHeight` to `2`, then `4`, then `8`. Recompute each time and keep `G-code Generator.layerHeight` at the same value. Watch `layers` change.
2. Set `wallCount` to `1`, then `3`. Look at `preview` in the viewport and count the concentric rings per layer.
3. Set `spacing` to `6`, then `15`, with `wallCount = 3`. Watch how far apart the wall rings sit. The value of `spacing` must be greater than 0; otherwise the component raises a hard error.

Read the output: `preview` from `Lugworm G-code Generator` shows the rings per layer.

Check yourself: doubling `layerHeight` should roughly halve `layers`. With `wallCount = 3`, three distinct concentric curves per layer should be visible, spaced by `spacing`.

4 Session 2: Infill patterns side by side

Grasshopper concept: sweeping an integer slider through named options; comparing outputs across runs.

Goal: see the practical difference between the four infill types, and learn to read `travels` and `time` as an indicator of print-quality risk.

Try this: on `Slice`, set `infill` to `1` (zigzag), recompute, and note the extent of `Lugworm G-code Generator.travels` and the value of `time`. Repeat for `2` (concentric), `3` (spiral), and `4` (fermat).

Read the output: compare the four runs. Does `time` go up or down? Do the `travels` curves become sparser? Run `Print Stats` (**5 Analyze**), connected to `G-code Generator.gcode`, and compare the `timeMin` and `travelLen` outputs across the four infill choices.

Check yourself: fermat and spiral should show noticeably fewer and shorter travel moves than zigzag and concentric, because they are built as continuous space-filling paths rather than as many separate rings or lines.

5 Session 3: Vase mode (spiralize)

Grasshopper concept: Boolean and menu toggles as mode switches.

Goal: understand `spiralize` as a special case that requires one closed contour per layer, and see what happens when a shape does not qualify.

Build: on `Test Geometry`, switch the dropdown to **Vase** or **TwistedVase** and set `height`, `radius`, and the profile inputs (`waveAmp`, `waveCount`, `twistDeg`) as preferred. Set `Lugworm Slice.wallCount = 1` and `infill = 0`.

Try this: enable the **Spiralize** menu toggle on `Slice` and recompute.

Read the output: `Slice.info` should confirm that spiralize ran.

Check yourself: if `info` reports instead that spiralize could not run because a layer does not reduce to one closed contour, simplify the profile (lower `waveAmp` or `twistDeg`) until every horizontal slice is a single closed loop.

6 Session 4: Seam control

Grasshopper concept: a second-stage component (`Lugworm Place`) transforming the tree of a first stage; visualising a point tree (`seams`).

Goal: see how `seamMode` changes where the start and stop point of each layer sits.

Build: reuse the cylinder from Session 1 (planar, several closed layers).

Try this: cycle `Place.seamMode` through `0` (Keep), `1` (Align, with a few `seamAngle` values), `2` (Stagger, with `seamAngle = 15`), `3` (Nearest, with a `seamPoint` placed to one side), and, optionally,

`4` (Guide, with a `seamCurve`) and `5` (Track-below, which keeps the seam aligned with the layer beneath rather than with a fixed target).

Read the output: display `Place.seams` as points and look from above.

Check yourself: mode `1` lines all seam points up in a column, mode `2` rotates them by `seamAngle` per layer, and mode `3` clusters them near the `seamPoint`. If nothing changes, confirm that `seamAngle`, `seamPoint`, or `seamCurve` is being changed, not only `seamMode`.

7 Session 5: Flow and pressure basics

Grasshopper concept: reading raw G-code text as a debugging tool.

Goal: learn where the flow and pressure inputs act by finding them directly in the `gcode` output.

Try this: on a part with `wallCount` of 2 or more, so that both contour ends inside a branch and travel moves are available for inspection:

1. Set `pressureOffset = 1` and `retractSpeed = 5`. Find a retract and prime pair in the `gcode` output and confirm that the pull-back distance and the feed rate match.
2. Set `coast = 1`. Find a contour end before a travel move and confirm that extrusion stops slightly early.
3. Set `leadIn = 3` and `leadInBoost = 1.2`. Look for the flow ramp at the start of a printed segment.
4. Set `minLayerTime = 5` and check whether the feed rates of any layer are scaled down as a result.

Read the output: point to the specific lines in `gcode` that changed for each setting.

Check yourself: if a setting seems to change nothing, check whether the relevant toggle group (`advance` or `retractaccel` for firmware hand-off features) is enabled. Several inputs only take effect once their group is switched on.

8 Session 6: Overhang analysis

Grasshopper concept: reading a warning threshold as a design signal rather than as an error to dismiss.

Goal: use `Overhang` (**5 Analyze**) to find where a shape is likely to be unprintable, and understand that the thresholds are placeholders, not calibrated values.

Build: a shape with a real overhang, for example the **Frustum** of `Test Geometry` flipped so that the shape narrows going down, or a **TwistedVase** with a large `waveAmp`.

Try this: connect the sliced or placed contours to `Overhang` and recompute (the component is Heavy). Look at the colour-coded preview and the `angles` output.

Check yourself: segments beyond `warnDeg` (default 25°) should be orange, segments beyond `failDeg` (default 45°) red. As *reference.tex* §4.1 explains, both numbers are uncalibrated placeholders. The exercise is about reading the analysis, not about trusting the specific angles.

9 Session 7: Print Stats as a pre-flight check

Grasshopper concept: treating a summary output as a gate before a long operation.

Goal: become comfortable reading `Print Stats` before a file is ever sent to a machine.

Try this: connect `G-code Generator.gcode` to `Print Stats`. Set `ramDiameter` to the value of the G-code Generator, so that the volume is computed from the actual E values rather than from the geometric fallback. Compare `timeMin`, `volumeCm3`, and `massG` with rough hand estimates for the size of the part.

Check yourself: deliberately mismatch `Lugworm Slice.layerHeight` and `G-code Generator.layerHeight` and watch `volumeCm3` become implausible. The exercise shows why the two values must match.

10 Session 8: Non-planar modes (advanced)

Grasshopper concept: reading a warning as a design constraint rather than as a bug to work around.

Part A, conical (`mode = 1`): on a shape with a shallow dome or taper, set `mode = 1`, `coneAngle = 20`, and `nozzleClear = 25`. If `info` warns that the nozzle clearance is exceeded, lower `coneAngle` until the warning disappears. Do not raise `nozzleClear` instead: that hides a real collision risk rather than resolving the risk (see *reference.tex* §2.1).

Part B, variable layer height: slice the same shape again with `coneAngle = 0` and with the **Adaptive** group on. Enable the `varh` group on `G-code Generator` in both cases and compare the reported volume with and without it. Neither adaptive nor conical layers keep a constant vertical spacing, so the extruded amount is only nominal until `varh` is on.

Check yourself: open a definition saved with the retired heat mode (`mode = 2`, removed in 4.0.0) and confirm that `Slice` reports "heat mode was retired in 4.0.0 - sliced planar" instead of failing.

11 Session 9: Robot simulation (dry run)

Grasshopper concept: treating a hard component error as non-negotiable, unlike a warning.

Goal: learn to validate a print on the robot before the print runs on real hardware.

Try this:

1. Place an `Robot Definition` (**4 Robot**) and select the profile `potterbot-scara-v4` (or enter `"scara"` in a text panel).
2. Connect the definition and `G-code Generator.gcode` to `Robot Simulation` (**4 Robot**). Scrub `sim` from `0` to `1` and watch the simulated arm trace the print. The printed layer and the travel moves are drawn in different colours.
3. Deliberately place the part (with `Place.printCenter`) somewhere unreasonable, far from the bed origin, and recompute.
4. Switch the definition to `none` and note that the component becomes a plain toolpath viewer: no arm, no reach checks, and `reachOK` true.

Read the output: watch `reachOK`, `problems`, `messages`, and the message balloon of the component.

Check yourself: an out-of-reach placement should set `reachOK` to false, fill `problems`, and raise a red error on the component ("targets are outside robot limits"). Move the part back to a sensible position and confirm that all three clear.

12 Session 10: A first pass through calibration (DoE)

Grasshopper concept: treating parameter tuning as a designed experiment instead of trial and error.

Goal: run one small but complete pass through the calibration loop without a printer, using invented measurements to see how the pieces fit together.

Try this:

1. `DoE Design`: enter two or three factors (for example `printSpeed` and `pressureOffset`) with reasonable low and high bounds, and choose the design `"BBD"`. Export the run table.
2. Invent a plausible quality score for each run (or estimate one from the G-code text of Session 5) and enter the scores into the exported table.
3. `RSM Evaluate`: point `tablePath` at the completed table, take `factorNames` from the DoE Design output, and set `model = "Quadratic"`. Check the reported R^2 and adjusted R^2. With invented data the fit will probably be poor, and that is the point: this is what a weak-fit warning looks like.
4. `Desirability`: connect `models`, set a goal string such as `"max"` for the quality score, and read the recommended settings.

Check yourself: explain in your own words why a real calibration campaign needs measured prints rather than invented numbers. This session is about the mechanics, not about a real result.

13 Optional exercises

- Compare the `arcs` toggle group on and off on a curved part and count the G-code lines before and after.
- Turn on `skirt` and find the purge ring in `preview`.
- Turn on `floors` with a few base layers and find where the behaviour changes in the `gcode` output.
- Place an `Print Bed`, connect the bed to `Place` and `Robot Simulation`, and confirm that the part lands on the print centre of the bed without setting `printCenter` by hand.
- Connect the robot definition to `Print Stats` and compare the Cartesian time estimate with the joint-limited estimate near the edge of the SCARA reach.
- Use `G-code Restart` to cut a print at 50 % and inspect the `printed` and `remaining` curve split.

14 Final project (printed together)

Once every session is complete, design one shared part that exercises at least **four** of the following, and bring the Grasshopper file to the final session:

- A non-trivial `infill` choice, justified by the shape.
- A deliberate `seamMode` choice, justified by where the seam is least visible.

- At least one pressure or flow setting tuned beyond the default (`pressureOffset`, `coast`, `leadIn`).
- Adaptive layers or conical mode, justified by a curved or sloped feature, with the matching `varh` setting where applicable.
- A pass through `Overhang` and `Print Stats` with no unaddressed red segments and a plausible time estimate.
- For printing on the SCARA: a clean `Robot Simulation` validation with the matching `Robot Definition` connected, `reachOK` true, and no red error.

Bring the `info` panels, the `Print Stats` report, and the robot validation where applicable. The group reviews them together, and the final files are printed on the shared machine as the course capstone.